

AAAAT Product Summary

What AAAAT is

AAAAT, Agent-Agnostic Auto Application Tracker, is a local-first, provider-agnostic job application tracker and artifact generator.

It is private local software for managing candidatures, recruiter interactions, profile/CV data, generated documents, and agent-assistable tasks. It gives the user a fast operational dashboard and gives external LLM agents a constrained task interface for drafting, enrichment, inference, and document preparation.

AAAAT is not an LLM provider wrapper. It is not an agent runtime. It is not an agent orchestrator. It does not require OpenAI, Anthropic, Gemini, Ollama, Codex, Claude Desktop, Cursor, Continue, Aider, or any other specific model or tool.

AAAAT owns the private application data, local dashboard, validation layer, task queue, template rendering, and artifact provenance. External agents own reasoning and generation.

Product promise

AAAAT lets a user manage job applications locally, prepare recruiter calls quickly, generate per-application artifacts, and use whichever LLM agent they already prefer without giving that agent broad access to the full private job-search database.

The user can also operate AAAAT manually without any agent.

Core design principles

1. Local ownership

AAAAT stores private data locally by default. The normal private storage location is `.private/` or another explicitly configured local path.

Private data includes candidatures, raw job descriptions, application forms, recruiter notes, profile/CV data, generated cover letters, generated CV variants, offers, salary expectations, and private artifacts.

Public examples and static demos must use fake data only.

2. Provider agnosticism

AAAAT does not ask for model provider, model URL, API key, model name, or cloud/local model settings during core setup.

The product should work with human-only usage, CLI usage, dashboard usage, and external agent usage. Any agent integration is passive and optional.

3. Deterministic ownership boundary

AAAAT owns deterministic operations:

- storing data;
- validating inputs;
- deciding which tasks exist;
- exposing only allowed task context;
- saving agent outputs;
- applying reviewed results;
- rendering templates;
- tracking generated artifact provenance;
- enforcing read-only/static/full modes.

Agents own non-deterministic work:

- inferring missing fields;
- summarizing job offers;
- drafting cover letters;
- drafting form responses;
- suggesting keyword definitions;
- preparing recruiter call notes;
- proposing CV adaptations;
- explaining rationale.

Agents propose. AAAAT records, validates, renders, and applies.

4. Task-scoped agent access

Agents must not receive classic CRUD access to candidatures, applications, dashboard payloads, profile facts, variables, or arbitrary search results.

The aggregate relation between one user and many candidatures is private data. A list of companies, roles, stages, dates, locations, salary expectations, generated documents, and outcomes can reveal job-search strategy, financial pressure, seniority, relocation plans, and career intent.

Therefore, agents should work through a task mailbox / capability membrane:

1. AAAAT creates or queues a task deterministically.
2. The user or local scheduler decides what is available.
3. The agent sees only pending task envelopes.
4. The agent requests context only for one specific task.
5. AAAAT builds a narrow context bundle for that task.
6. The agent submits a result to that task.

7. AAAAT stores the result with provenance.
8. AAAAT applies it only through deterministic review/apply logic.

The agent is a worker consuming capability-scoped jobs, not a client browsing the user's job-search database.

5. Human dashboard plane vs. agent plane

AAAAT has two distinct access planes.

The human/local dashboard plane may expose rich local CRUD because it is the user's own operational UI. It can show candidatures, search, profile data, notes, tasks, documents, rendered values, and editable fields according to the selected mode.

The agent plane is task-scoped only. It should expose task envelopes, task context, task claiming/releasing, and task result submission. It should not expose list-all candidature routes, raw variables, raw profile facts, dashboard payloads, arbitrary search, or generic patch routes.

6. Privacy by resolution, not by convention

Private values should be represented as variables or structured profile facts and resolved only by AAAAT according to context.

Examples:

```
profile.display_name
profile.email
profile.phone
profile.location
profile.summary.default
profile.skills.backend
profile.experience.software_projects
application.company
application.role
artifact.cover_letter.body
```

Depending on the surface, AAAAT may expose:

- raw value;
- redacted value;
- summarized value;
- placeholder only;
- denied.

Local rendering may resolve private values. Agent contexts should receive the minimum safe representation required for the task.

7. Artifact provenance and review

Generated artifacts are first-class records, not anonymous files.

Each artifact should track:

- candidature/application id;
- artifact type;
- path;
- label;
- created timestamp;
- source context;
- generating agent name, if available;
- generating runtime/model/provider, if voluntarily reported;
- review state;
- notes.

Artifact states are:

```
draft
reviewed
submitted
archived
```

AAAAT should not create unlimited artifact clutter. The current/final useful artifact should be shown first. Archived or older versions should remain available but not dominate the dashboard.

8. Anonymized reusable templates

AAAAT templates must not hardcode the real user's identity, CV content, recruiter messages, or private data.

CV and cover-letter templates should be generic and use variables. Rendered local outputs may contain private values, but source templates should remain reusable and safe to commit.

9. Operational dashboard first

The dashboard is not a documentation site. It is an operational surface for job-search execution and recruiter calls.

It should answer quickly:

- What company is this?
- What role is this?
- What status is it in?
- What should I do next?

- What did I already send?
- What is my pitch?
- What risk should I avoid saying?
- What does this keyword mean?
- What artifacts exist?
- What questions should I ask?

The dashboard should avoid clutter, giant static lists, excessive drafts, ambiguous artifact names, deep navigation, and decorative UI that hides operational information.

10. Clear operating modes

AAAAT should support these user-facing modes:

Full local mode:

- local server and dashboard;
- editable candidatures;
- raw intake;
- task controls;
- document rendering;
- private data visible where appropriate.

Local read-only mode:

- same local data;
- no raw intake or write controls;
- useful for recruiter calls and review sessions.

Static public demo mode:

- generated from fake demo data;
- no backend;
- no private data;
- no raw intake;
- no write controls;
- demonstrates the product safely.

Direct manual mode:

- user can operate without an external agent.

Agent access mode:

- external agents interact only through constrained task/capability interfaces.

Main product objects

Candidature

A candidature is the central product object. It represents one job opportunity or application process.

It can include company, role, status, priority, source, raw offer text, raw application form, description, salary expectation, dates, company research, strengths, risks to avoid, questions to ask, tech stack, keywords, notes, todos, tasks, text blobs, generated documents, and artifact references.

Task

A task is the controlled unit of agent work.

Tasks are queued by AAAAT or the user. A task points to a narrow instruction and allows an agent to produce a specific result without browsing the whole database.

Examples:

- infer missing candidature fields;
- research company context;
- define missing keyword;
- draft cover letter;
- adapt CV;
- draft form responses;
- prepare recruiter call;
- generate interview guide.

Keyword

A keyword is a global glossary term with optional aliases and notes. Keywords can be detected from raw inputs, attached to candidatures, selected in the dashboard, and used to trigger definition tasks.

Profile fact / variable

Profile data is used for CVs, cover letters, market positioning, and agent context. It must be classified by visibility and exposure so AAAAT can decide whether a value is raw, summarized, anonymized, redacted, placeholder-only, or denied.

Artifact

An artifact is a generated or attached file with state and provenance. Examples include CV variants, cover letters, recruiter messages, form answer sheets, and interview preparation documents.

Non-goals

AAAAT should not become:

- a cloud job-search platform;
- an LLM provider wrapper;
- a chat application;
- an agent runtime;
- a model router;
- a SaaS synchronization service;
- a CRM clone;
- a heavy frontend application;
- a database-server application;
- a system that gives agents broad unrestricted access to the user's job-search data.

Design summary

AAAAT is the local private control layer for job applications.

It lets the user manage candidatures, profile data, tasks, documents, and recruiter preparation in one local dashboard. It lets agents help, but only through narrow task-scoped capabilities. It resolves private values only where appropriate, records provenance for generated outputs, and keeps the final decision and submission flow under user control.